EE542 Lab 8: CNN Inference on Network Packets with NVIDIA DOCA

Huacong Tang
University of Southern California

Xiao Yang
University of Southern California

Jiachen Lu
University of Southern California

Code Demo Video

1 Setup

Table 1 shows the machines.

Part	Machine	Software
1–5	RunPod: L4 23 GB, EPYC 9254, Ubuntu 22.04.5	driver 580.159.04, CUDA 13.0, toolkit 12.6, PyTorch 2.14.1+cu130, torchvision 0.29.1, TensorRT 11.4.0.106, ONNX 1.23.2, DPDK 21.11.9, gcc 11.4
6	RunPod: L4, DOCA 3.5.0 image as container	driver 595.91.07
6	AWS `g6.xlarge` VM: L4	driver 595 line, Docker 29.9, NVIDIA Container Toolkit
7	Two CloudLab d7525: A30, ConnectX-6 200 GbE, EPYC 7302, 125 GB, Ubuntu 24.04.5, kernel 6.8.0-147	driver 580.196.07, CUDA 13.0, GDRCopy 2.5.2, GPUNetIO `b2b7a29`

Table 1: Machines and software.

2 TensorRT inference from GPU memory

Export and engines. We exported ResNet-18 to ONNX in FP32 and FP16 with a dynamic batch size. Keep `.onnx.data` with the `.onnx` file and load with `parser.parse_from_file()`. The FP16 engine needs the FP16 ONNX file. Each engine covers batch 1 to 32 with OPT=8. Inference runs from GPU buffers on a dedicated CUDA stream.

Measurements. FP16 is 2.7–4.2× faster than PyTorch FP32 (Table 2, Figure 1). Over 1,024 inputs, the maximum logit difference is 0.011 (FP32) and 0.035 (FP16), with 100% top-1 agreement. Both frameworks use TF32 for some FP32 math.

Why TensorRT FP32 loses to PyTorch at batch 1. TensorRT selects kernels for the OPT shape (Table 3, Figure 2). At batch 1, FP32 with OPT=1 takes 0.783 ms, less than PyTorch (0.992 ms). With OPT=8 it takes 1.218 ms. No single profile is fastest at all batch sizes.

Implementation	Batch	Latency (ms)	Speedup	Images/s
PyTorch FP32	1	0.992	1.00×	1,008
	8	2.636	1.00×	3,035
	32	10.259	1.00×	3,119
TensorRT FP32	1	1.218	0.81×	821
	8	2.229	1.18×	3,589
	32	8.528	1.20×	3,753
TensorRT FP16	1	0.364	2.73×	2,747
	8	0.626	**4.21**×	**12,789**
	32	2.602	3.94×	12,297

Table 2: ResNet-18 on the L4, OPT=8, mean of 500 runs after 50 warm-ups. Speedup is PyTorch time divided by TensorRT time.

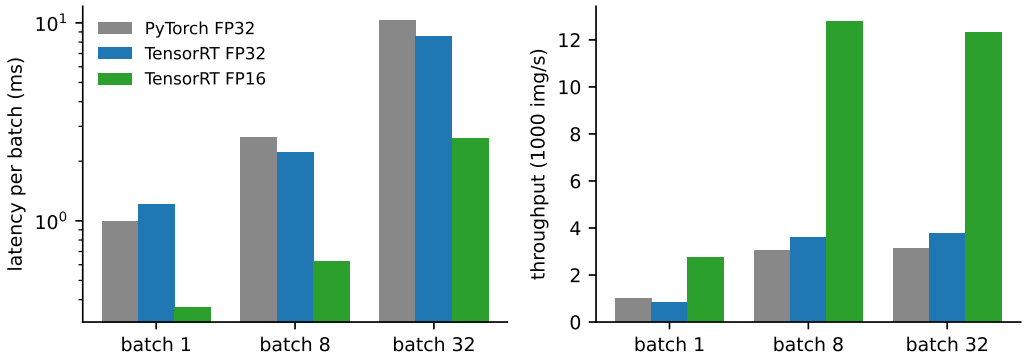


Figure 1: Latency (left, log scale) and throughput (right) per batch size.

	FP32, latency (ms)			FP16, latency (ms)		
Engine	batch 1	batch 8	batch 32	batch 1	batch 8	batch 32
OPT=1	**0.783**	3.473	13.941	**0.207**	1.025	4.104
OPT=8	1.215	2.266	8.657	0.375	**0.592**	2.624
OPT=32	1.193	**2.017**	**6.478**	0.754	0.939	**2.626**

Table 3: Fixed-OPT engines. Bold marks the column minimum.

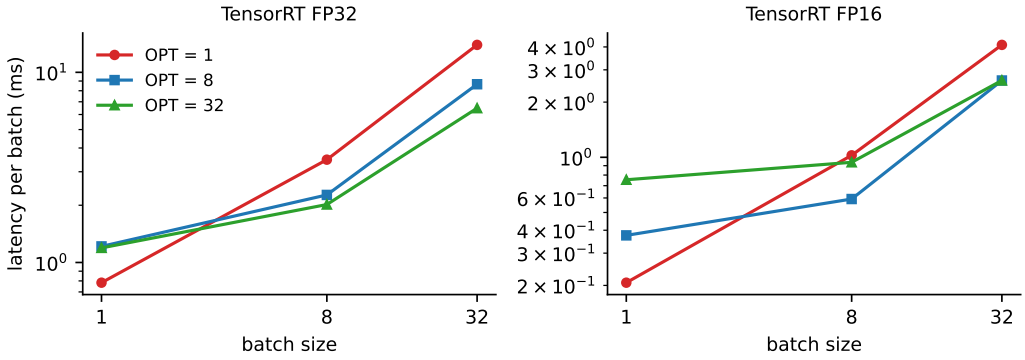


Figure 2: Latency per batch for each OPT. The curves cross.

3 Images as network packets

One $224 \times 224 \times 3$ image has 150,528 bytes and needs $\lceil 150{,}528/1{,}400 \rceil = 108$ packets. The last packet has 728 data bytes. Each payload has a 22-byte header (frame, segment, offset, length, size) and 1,400 data bytes. The IP datagram has 1,450 bytes, which fits a 1,500-byte MTU. A Scapy generator writes 16 images (1,728 packets) to `images.pcap`. It can shuffle, duplicate, and add noise.

4 Receiving the packets with DPDK

The receiver uses the DPDK pcap port, so it needs no network hardware. It drops non-IPv4 packets and other ports, and copies each chunk to its byte offset. One bitmap per frame removes the effect of packet order and stops duplicates. The program stops after 500 ms with no packets, or at `-expect N` frames.

All runs rebuild 16/16 images byte for byte (Table 4). The hostile run drops exactly the 186 added packets. Port counters stay at zero. The pcap port receives 0.74–0.80 Mpps, about 7,000 images/s, less than the 12,789 images/s of the FP16 engine. So the receive path is the bottleneck.

Capture file	Received	Accepted	Dropped (reason)	Complete	Byte-exact
In order	1,728	1,728	0	16	16/16
Shuffled	1,728	1,728	0	16	16/16
Shuffled + hostile	1,914	1,728	186 (86 dup, 50 non-IPv4, 50 port)	16	16/16

Table 4: Receiver runs over 16 images $\times$ 108 packets. “Hostile” adds 5% duplicates, 50 foreign-port packets, and 50 ARP packets.

5 Classifying the received images on the GPU

One pinned copy moves the images to the GPU. A CUDA preprocess converts HWC uint8 to normalized CHW float, and the FP16 engine classifies them. All 16 images get the same class as their originals. The model is correct on 13/16 labels. The three errors (tabby, fire engine, banana) also occur on the originals.

GPUNetIO removes this copy, the DPDK receive, and the `memcpy` reassembly. Table 5 shows the copy cost. Its share grows with batch size (Figure 3).

Images	Pinned copy	With launch and sync (median)	Pageable	Share of GPU-side time
1	12.6 µs, 11.9 GB/s	18.5 µs	6.9 GB/s	3.1% of 402 µs
16	182 µs, 13.3 GB/s	188.5 µs	11.3 GB/s	11.7% of 1,547 µs

Table 5: Host-to-GPU copy cost.

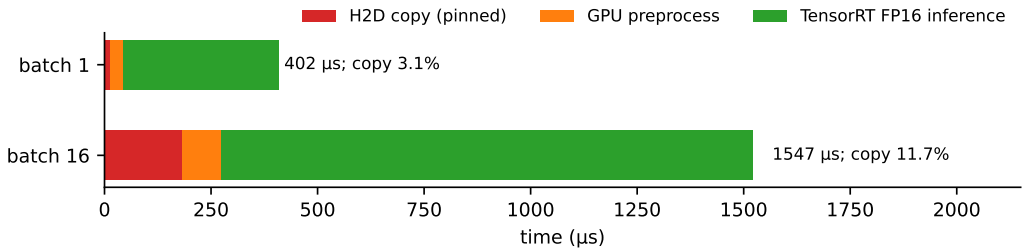


Figure 3: GPU-side time per batch: pinned copy, CUDA preprocess, FP16 engine.

6 NVIDIA DOCA container and GPUNetIO

We added `sm_89` for the L4 to `meson.build`. `cuobjdump` shows `sm_75/80/89/90/100`. Table 6 shows the results. The guide recommends ConnectX-6 Dx or later, and GPUNetIO needs a GPUDirect RDMA path (dmabuf or `nvidia-peermem`). So neither instance can run GPUNetIO.

Check	RunPod container	AWS VM
`docker run ... nvidia-smi -L`	not run (already in container)	`GPU 0: NVIDIA L4`
`doca_caps`	DOCA 3.5.0098, 21 libraries	same
`doca_caps -list-devs`	"No DOCA device was found"	same
`simple_receive -e 0/1/2`	"Matching device not found"	same
`verbs_write_lat`	"Failed to get RDMA devices list"	same
NIC	ConnectX-4 Lx at `41:00.0`	ENA only, no vendor `15b3`
Reason	no `/dev/infiniband`, no `SYS_ADMIN`, no `NET_ADMIN`, netdev in other namespace	no physical RDMA NIC

Libraries: common, aes_gcm, argp, comch, compress, devemu_pci, dma, dpa, dpdk_bridge, erasure_coding, eth, flow, gpunetio, mgmt, pcc, rdma, sha, telemetry, telemetry_exporter, urom, verbs.

Table 6: Part 6 results.

Three build notes. On a driver-595 host, remove `cuda-13/compat` from `LD_LIBRARY_PATH`, or CUDA initialization fails. Build GPUNetIO with `CUDA_ARCH=89`, and run `make install` before `make install_examples`. The `libgdrapi.so.2` message is harmless.

7 GPU-initiated RDMA on CloudLab

Two bare-metal Dell R7525 nodes share one RoCE v2 LAN (Table 1, profile). After reboot, both show IOMMU off, ACS off, 2048 hugepages, `gdrdrv` loaded, and persistence on. One reboot applied the driver and IOMMU changes, so Step 7.5 ran with IOMMU off.

	CloudLab d7525	AWS `g6.xlarge`	RunPod L4 pod
GPU	A30 at `25:00.0`	L4 at `31:00.0`	L4
NIC	ConnectX-6 at `41:00.0`	ENA at `27:00.0`	ConnectX-4 Lx at `41:00.0`
NIC firmware	20.36 (node0), 20.41 (node1)	not applicable	not captured
RDMA device	`rocep65s0`	none	hidden from container
GPU–NIC path	`NODE`, same NUMA node	no RDMA NIC	not captured

Table 7: Step 7.4 against the Part 6 instances. Both devices run PCIe 16 GT/s ×16. The guide recommends PIX or PXB and says to avoid `NODE`.

`gpunetio_simple_receive`. The sample gets further than in the cloud (Figure 4), then stops at "Failed to get hws cap" in all `-e` modes on both nodes. The firmware rejects the WQE-based flow table query (`BAD_PARAM`, syndrome `0x77ad2a`). The guide recommends ConnectX-6 Dx or later, and this card is a ConnectX-6. The `compat` libcuda also blocked the first attempt, as in Section 6.

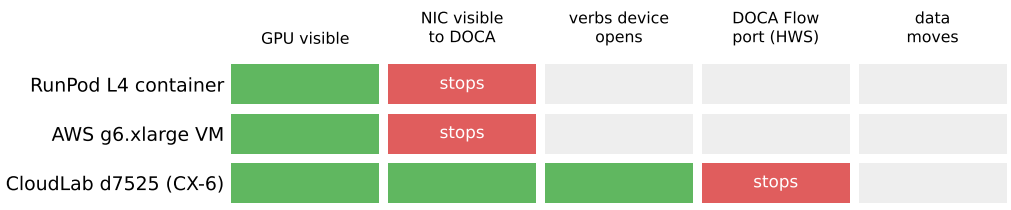


Figure 4: How far `gpunetio_simple_receive` gets.

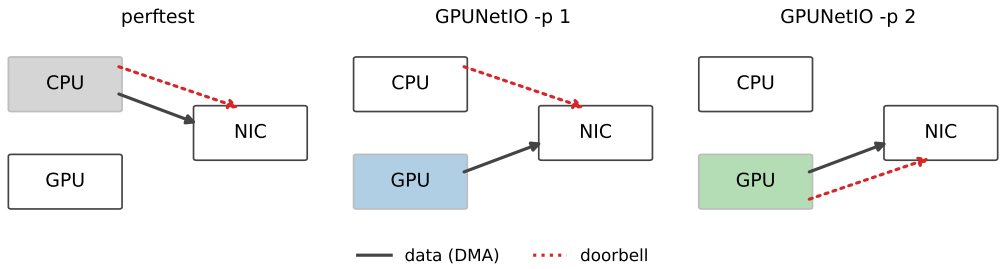


Figure 5: Data location and doorbell. On the GPU paths, a CUDA kernel builds the requests.

Measurement	GPUNetIO `-p 1`	GPUNetIO `-p 2` (root)	perftest (CPU)
One-way latency, smallest message (µs)	8.37	8.23	2.52
Bandwidth at 64 KB (Gbps)	47.2	49.7	108.4

Table 8: Means of three rounds. GPUNetIO: `write_lat` and `put_bw`, 1 B. perftest: 2 B with inline data, t_{typical}, average bandwidth.

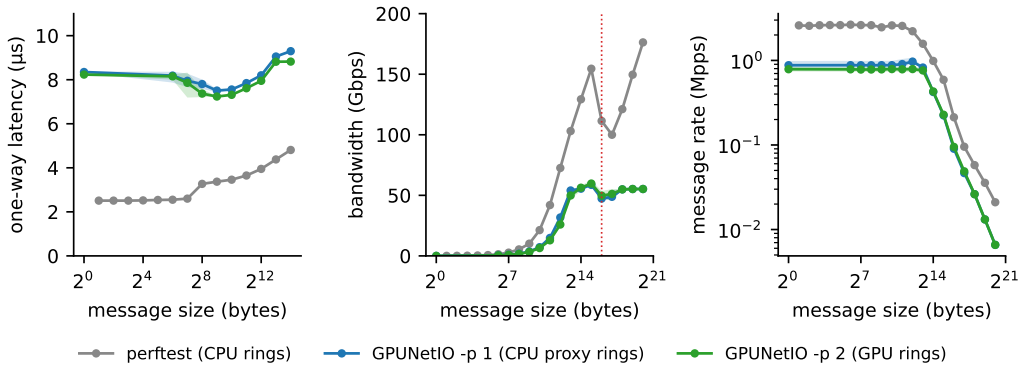


Figure 6: GPUNetIO: three-round means, range shaded, both as root. perftest: one `-a` run, CQ moderation 100. Dotted line: 64 KB.

GPU-to-GPU RDMA. The open-source examples do not use DOCA Flow, and they work (Figure 5, Table 8, Figure 6).

perftest is better in both rows. It keeps data in host memory, and the CPU tells the NIC to send. The GPU doorbell cuts less than 0.5 µs of the 5.9 µs gap and does not change the 55 Gbps plateau, so the doorbell is not the limit. We did not measure the rest of the gap. The `NODE` path is one possible cause of the plateau. `write_bw` (GID 0) agrees with `put_bw`: 54–59 Gbps from 8 KB up, outside the dip. All server data checks pass.

What the GPU needs to ring the doorbell. The GPU needs the NIC doorbell page (UAR) mapped into GPU-visible memory. As a normal user, `-p 2` fails: `cudaHostRegister` error 800, "Can't export UAR to GPU", and in `dmesg` "permission denied, allowPeermapping=0". As root, all 10 `-p 2` runs finish with no proxy thread. So the barrier is a driver permission, not the hardware.

The 64 KB dip. GPU and CPU writes both lose bandwidth near 64 KB (Figure 7). Receiver drops are zero up to 32 KB and start at 64 KB. We did not find why. We probed node1 to node0 only.

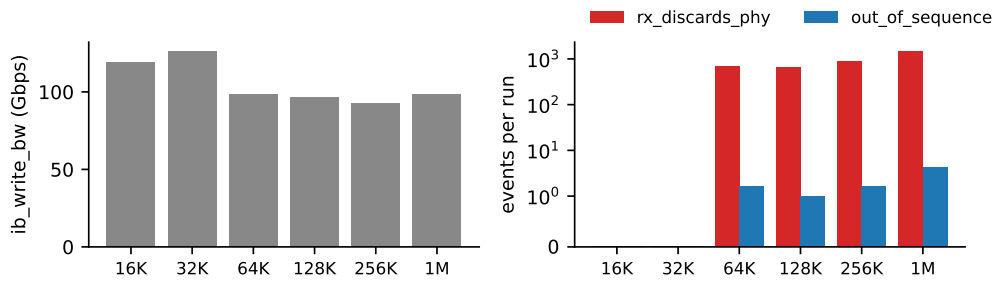


Figure 7: Three runs per size (bytes), CQ moderation 1, node1 to node0. All drops are on node0.